

Reinforcement Learning: Value Functions and Value Iteration

Stanford Lecture Notes

Abstract

These notes cover value functions, Bellman equations, and algorithms for solving MDPs. Topics include the value function for a policy (V_π), the optimal value function (V^*), optimal policies (π^*), and the value iteration algorithm for computing optimal policies.

Contents

1	Recap: Markov Decision Process (MDP)	3
1.1	MDP Definition	3
1.2	MDP Execution	3
1.3	Policy	4
1.4	Challenge: Finding the Optimal Policy	4
2	Value Functions	4
2.1	Value Function for a Policy: V_π	5
2.1.1	Example: Value Function for a Specific Policy	5
3	Bellman's Equation for V_π	5
3.1	Intuition: Decomposing Expected Payoff	5
3.2	Bellman's Equation for Policy Evaluation	5
3.3	Solving for V_π Using Bellman's Equation	6
3.3.1	Concrete Example	6
3.3.2	Implementation	7
4	Optimal Value Function and Optimal Policy	7
4.1	Optimal Value Function: V^*	7
4.2	Bellman's Equation for V^*	7
4.2.1	Intuition	7
4.3	Optimal Policy: π^*	8
4.4	Optimality Property	8
5	Value Iteration Algorithm	8
5.1	Strategy for Finding Optimal Policy	8
5.2	Value Iteration Algorithm	8
5.2.1	Algorithm Explanation	8
5.3	Implementation Details	9
5.3.1	Data Structure	9
5.3.2	Synchronous vs. Asynchronous Updates	9

5.3.3	Bellman Backup Operator	10
5.4	Convergence Properties	10
5.5	Handling Absorbing States	10
6	Example: Computing Optimal Policy	11
6.1	Running Value Iteration	11
6.2	Extracting the Optimal Policy	11
7	Summary	11
7.1	Key Concepts	11
7.2	Algorithm Workflow	12
7.3	Extensions	12
8	Additional Notes	12
8.1	Number of States in Grid World	12
8.2	Practical Considerations	13

1 Recap: Markov Decision Process (MDP)

1.1 MDP Definition

An MDP is a 5-tuple: $\langle S, A, P_{sa}, \gamma, R \rangle$

S Set of states (e.g., 11 states in the grid world)

A Set of actions (e.g., North, South, East, West)

P_{sa} State transition probabilities

γ Discount factor, typically $\gamma \approx 0.99$

R Reward function: $R : S \rightarrow \mathbb{R}$

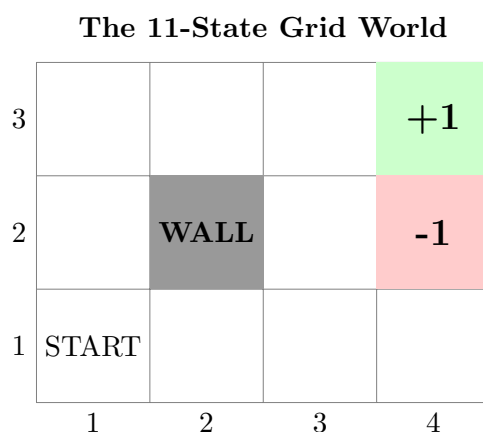


Figure 1: The standard Grid World environment. The agent starts at (1,1). (2,2) is an obstacle. (4,3) and (4,2) are terminal states with rewards.

1.2 MDP Execution

The agent interacts with the environment over time:

1. Start in initial state s_0
2. Choose action a_0
3. Transition to state $s_1 \sim P_{s_0 a_0}(\cdot)$
4. Choose action a_1
5. Transition to state $s_2 \sim P_{s_1 a_1}(\cdot)$
6. Continue: $s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow \dots$

Total Payoff:

$$\sum_{t=0}^{\infty} \gamma^t R(s_t)$$

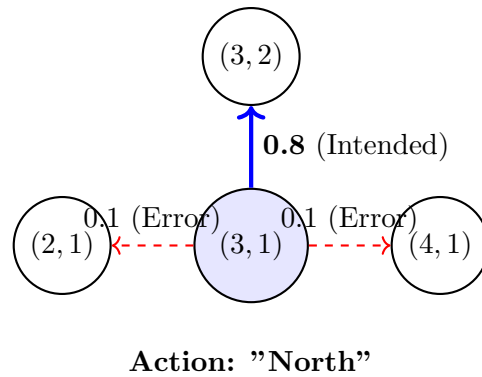


Figure 2: Visualization of P_{sa} . When the agent tries to move **North**, there is an 80% chance it succeeds, and a 10% chance it slips West or East.

1.3 Policy

A **policy** π is a function mapping states to actions:

$$\pi : S \rightarrow A$$

Executing policy π : At each time step t , take action $a_t = \pi(s_t)$.

Goal: Find policy π that maximizes expected total discounted payoff.

1.4 Challenge: Finding the Optimal Policy

Problem: The number of possible policies is exponentially large.

For an MDP with $|S|$ states and $|A|$ actions per state:

$$\text{Number of policies} = |A|^{|S|}$$

For the 11-state grid world with 4 actions: $4^{11} \approx 4.2$ million policies.

Question: How do we efficiently find the best policy without enumerating all possibilities?

2 Value Functions

To develop an algorithm for computing optimal policies, we define three key concepts:

1. V_π — Value function for policy π
2. V^* — Optimal value function
3. π^* — Optimal policy

2.1 Value Function for a Policy: V_π

Definition: Value Function for Policy π

For a policy π , the value function $V_\pi : S \rightarrow \mathbb{R}$ is defined as:

$$V_\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t) \mid \pi, s_0 = s \right]$$

Interpretation: $V_\pi(s)$ is the expected total discounted payoff when starting in state s and executing policy π .

2.1.1 Example: Value Function for a Specific Policy

Consider a (suboptimal) policy in the grid world. The value function V_π assigns a numerical value to each state:

- States that lead efficiently to the +1 reward: positive values
- States that lead to the -1 penalty: negative values
- Example: $V_\pi(1,1) = -0.88$ (leads toward the bad state)

Key Insight: Different policies yield different value functions. A good policy has higher values across states.

3 Bellman's Equation for V_π

3.1 Intuition: Decomposing Expected Payoff

Consider starting in state $s_0 = s$:

$$\begin{aligned} V_\pi(s) &= \mathbb{E} \left[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots \mid \pi, s_0 = s \right] \\ &= R(s_0) + \gamma \mathbb{E} \left[R(s_1) + \gamma R(s_2) + \gamma^2 R(s_3) + \cdots \mid \pi, s_0 = s \right] \\ &= R(s) + \gamma \mathbb{E}_{s' \sim P_{s, \pi(s)}} [V_\pi(s')] \end{aligned}$$

Components:

- $R(s)$ — **Immediate reward** for being in state s
- $\gamma \mathbb{E}[V_\pi(s')]$ — **Expected future rewards** (discounted)

3.2 Bellman's Equation for Policy Evaluation

Bellman's Equation for V_π

$$V_\pi(s) = R(s) + \gamma \sum_{s' \in S} P_{s, \pi(s)}(s') \cdot V_\pi(s')$$

where s' is the next state after taking action $\pi(s)$ in state s .

Interpretation:

- The value of state s equals the immediate reward plus the discounted expected value of the next state
- The next state s' is distributed according to $P_{s,\pi(s)}(\cdot)$

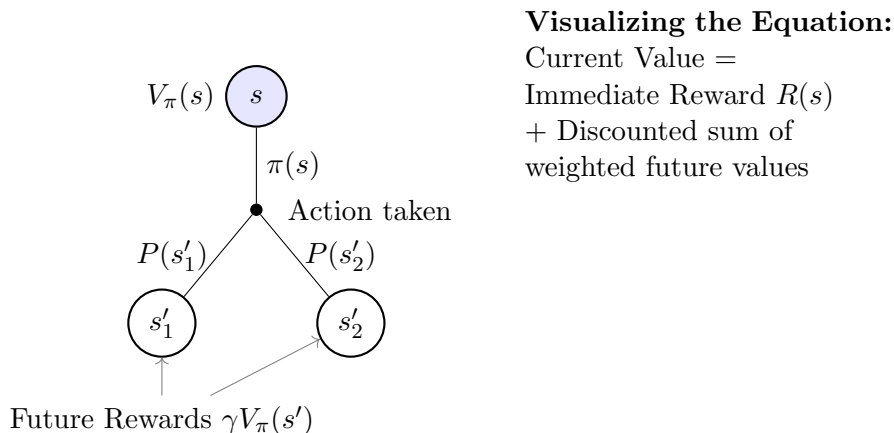


Figure 3: Bellman Backup Diagram for a fixed policy. We look ahead one step to average the values of possible next states.

3.3 Solving for V_π Using Bellman's Equation

Bellman's equation defines a **linear system of equations**:

For each state s , we have one equation:

$$V_\pi(s) = R(s) + \gamma \sum_{s'} P_{s,\pi(s)}(s') \cdot V_\pi(s')$$

3.3.1 Concrete Example

For state $(3, 1)$ in the grid world, suppose $\pi(3, 1) = \text{North}$:

$$\begin{aligned} V_\pi(3, 1) = R(3, 1) + \gamma & \left[0.8 \cdot V_\pi(3, 2) \right. \\ & + 0.1 \cdot V_\pi(2, 1) \\ & \left. + 0.1 \cdot V_\pi(4, 1) \right] \end{aligned}$$

For 11-state MDP:

- 11 equations (one per state)
- 11 unknowns: $V_\pi(s)$ for each state s
- **Solution:** Use linear algebra solver (e.g., matrix inversion)

3.3.2 Implementation

Create an 11-dimensional vector:

$$\mathbf{v} = \begin{bmatrix} V_\pi(1, 1) \\ V_\pi(1, 2) \\ \vdots \\ V_\pi(4, 3) \end{bmatrix}$$

Write Bellman's equations in matrix form:

$$\mathbf{v} = \mathbf{r} + \gamma \mathbf{P} \mathbf{v}$$

Solve for $\mathbf{v}$:

$$\mathbf{v} = (\mathbf{I} - \gamma \mathbf{P})^{-1} \mathbf{r}$$

4 Optimal Value Function and Optimal Policy

4.1 Optimal Value Function: V^*

Definition: Optimal Value Function

$$V^*(s) = \max_{\pi} V_{\pi}(s)$$

Interpretation: $V^*(s)$ is the maximum expected total payoff achievable from state s over all possible policies.

Key Point: V^* looks across all $|A|^{|S|}$ possible policies and takes the maximum value for each state.

4.2 Bellman's Equation for V^*

4.2.1 Intuition

Starting in state s , what is the best possible expected payoff?

1. Receive immediate reward $R(s)$
2. Choose some action a
3. Transition to next state $s' \sim P_{sa}(\cdot)$
4. Receive future payoff $V^*(s')$ (discounted by γ)

Question: Which action a should we choose?

Answer: The action that maximizes expected future payoff!

Bellman's Equation for Optimal Value Function

$$V^*(s) = R(s) + \max_{a \in A} \gamma \sum_{s' \in S} P_{sa}(s') \cdot V^*(s')$$

Interpretation: The optimal value equals the immediate reward plus the maximum (over all actions) of the expected discounted future value.

4.3 Optimal Policy: π^*

Given V^* , we can extract the optimal policy:

Definition: Optimal Policy

$$\pi^*(s) = \arg \max_{a \in A} \sum_{s' \in S} P_{sa}(s') \cdot V^*(s')$$

Note: The factor γ is omitted since it doesn't affect the argmax.

Interpretation: In each state s , the optimal policy chooses the action that maximizes expected future value.

4.4 Optimality Property

For all states s and all policies π :

$$V^*(s) = V_{\pi^*}(s) \geq V_{\pi}(s)$$

Meaning: The optimal policy π^* achieves the optimal value V^* in every state, and performs at least as well as any other policy in every state.

5 Value Iteration Algorithm

5.1 Strategy for Finding Optimal Policy

1. **Compute** V^* using an algorithm (value iteration)
2. **Extract** π^* using:

$$\pi^*(s) = \arg \max_a \sum_{s'} P_{sa}(s') \cdot V^*(s')$$

5.2 Value Iteration Algorithm

Value Iteration Algorithm

Initialize: For all $s \in S$, set $V(s) = 0$

Repeat until convergence:

$$V(s) := R(s) + \max_{a \in A} \gamma \sum_{s' \in S} P_{sa}(s') \cdot V(s') \quad \forall s \in S$$

Output: $V \approx V^*$ after convergence

5.2.1 Algorithm Explanation

- Start with initial value estimates $V(s) = 0$ for all states
- Repeatedly update values using Bellman's equation for V^*

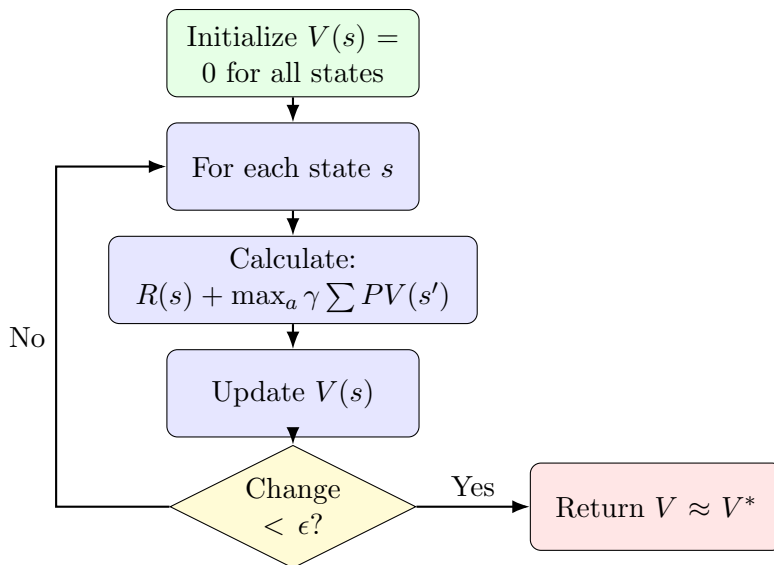


Figure 4: Flowchart of the Value Iteration Algorithm.

- Each update improves the value estimates
- Converges to optimal value function V^*

5.3 Implementation Details

5.3.1 Data Structure

Create an 11-dimensional vector to store values:

$$\mathbf{V} = [V(1, 1), V(1, 2), \dots, V(4, 3)]^T$$

Initialize: $V(s) = 0$ for all s .

5.3.2 Synchronous vs. Asynchronous Updates

Synchronous Update:

- Compute new values for all states using old values
- Update all values simultaneously
- $V^{\text{new}} = B(V^{\text{old}})$ where B is the Bellman backup operator

Asynchronous Update:

- Update values one at a time
- Use the most recent value estimates in subsequent computations
- Example: After updating $V(1, 1)$, use the new value when computing $V(1, 2)$

Note: Value iteration converges with either update scheme. Synchronous updates are more common because they vectorize better (efficient matrix operations).

5.3.3 Bellman Backup Operator

The update step can be written as:

$$\mathbf{V} \leftarrow B(\mathbf{V})$$

where B is the **Bellman backup operator**:

$$[B(\mathbf{V})](s) = R(s) + \max_a \gamma \sum_{s'} P_{sa}(s') \cdot V(s')$$

5.4 Convergence Properties

Convergence Guarantee

Value iteration converges to the optimal value function:

$$\lim_{k \rightarrow \infty} V^{(k)} = V^*$$

Convergence Rate: With discount factor γ , the error decreases by a factor of γ per iteration:

$$\|V^{(k+1)} - V^*\|_{\infty} \leq \gamma \|V^{(k)} - V^*\|_{\infty}$$

Rate of Convergence:

- $\gamma = 0.99$: converges in ~ 100 -200 iterations
- $\gamma = 0.9$: converges in ~ 10 -20 iterations

The algorithm converges **geometrically** (exponentially fast).

Details: See problem set for formal proof of convergence.

5.5 Handling Absorbing States

Problem: In the grid world, states (4,3) and (4,2) are absorbing — once reached, the episode ends.

Implementation Options:

1. **Option 1:** Set $P_{sa}(s') = 0$ for all $s' \neq s$ from absorbing states
 - State transitions to itself with probability 1
 - Simple to implement
2. **Option 2:** Add a 12th "terminal" state
 - Absorbing states transition to terminal state
 - Terminal state loops to itself with $R = 0$
 - More mathematically clean but less common

6 Example: Computing Optimal Policy

6.1 Running Value Iteration

After running value iteration on the 11-state grid world, we obtain:

$$V^* = [V^*(1,1), V^*(1,2), \dots, V^*(4,3)]$$

Example values (approximations):

- $V^*(2,1) = 0.75$
- $V^*(3,2) = 0.69$
- $V^*(3,1) = 0.71$
- $V^*(4,1) = 0.49$

6.2 Extracting the Optimal Policy

For state $(3,1)$, compute expected future reward for each action:

Action: West (left)

$$\begin{aligned} & \sum_{s'} P_{(3,1), \text{West}}(s') \cdot V^*(s') \\ &= 0.8 \cdot V^*(2,1) + 0.1 \cdot V^*(3,2) + 0.1 \cdot V^*(3,1) \\ &= 0.8 \cdot 0.75 + 0.1 \cdot 0.69 + 0.1 \cdot 0.71 \\ &= 0.740 \end{aligned}$$

Action: North

$$\begin{aligned} & \sum_{s'} P_{(3,1), \text{North}}(s') \cdot V^*(s') \\ &= 0.8 \cdot V^*(3,2) + 0.1 \cdot V^*(2,1) + 0.1 \cdot V^*(4,1) \\ &= 0.8 \cdot 0.69 + 0.1 \cdot 0.75 + 0.1 \cdot 0.49 \\ &= 0.676 \end{aligned}$$

Conclusion:

$$0.740 > 0.676 \implies \pi^*(3,1) = \text{West}$$

Similarly check East and South to confirm West is optimal.

7 Summary

7.1 Key Concepts

1. **Value Function** $V_\pi(s)$: Expected total payoff from state s under policy π
2. **Bellman's Equation (policy evaluation):**

$$V_\pi(s) = R(s) + \gamma \sum_{s'} P_{s, \pi(s)}(s') \cdot V_\pi(s')$$

3. **Optimal Value Function** $V^*(s)$: Maximum expected payoff achievable from state s

4. **Bellman's Equation (optimality)**:

$$V^*(s) = R(s) + \max_a \gamma \sum_{s'} P_{sa}(s') \cdot V^*(s')$$

5. **Optimal Policy** $\pi^*(s)$:

$$\pi^*(s) = \arg \max_a \sum_{s'} P_{sa}(s') \cdot V^*(s')$$

6. **Value Iteration**: Iteratively apply Bellman backup operator to compute V^*

7.2 Algorithm Workflow

Algorithm 1 Complete Solution for MDP

- 1: **Given:** MDP $\langle S, A, P_{sa}, \gamma, R \rangle$
- 2: **Step 1:** Run value iteration to compute V^*
- 3: **Step 2:** Extract optimal policy:

$$\pi^*(s) = \arg \max_a \sum_{s'} P_{sa}(s') \cdot V^*(s') \quad \forall s \in S$$

- 4: **Step 3:** Deploy robot using policy π^*
-

7.3 Extensions

Topics for future lectures:

- Continuous state spaces (discretization or function approximation)
- Learning state transition probabilities P_{sa} from data
- Model-free reinforcement learning (Q-learning, policy gradients)
- Applications to real-world robotics and control

8 Additional Notes

8.1 Number of States in Grid World

The example grid world has **11 states** (not 10):

- $4 \times 3 = 12$ cells total
- Minus 1 obstacle (wall)
- Equals 11 reachable states

8.2 Practical Considerations

- For small MDPs ($|S| \approx 10 - 100$), value iteration is very efficient
- For larger state spaces, approximations or sampling methods may be needed
- Discount factor γ affects both convergence speed and problem semantics
- Synchronous updates enable vectorization and GPU acceleration